Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет

по лабораторной работе №4
по дисциплине «Программирование на языке Java»
на тему: «Работа с файлами»
Вариант 6

Выполнили студенты группы 23ВВВ3:
Гуреев Д. Р.
Ларин Е. Д.
Приняли:
Юрова О. В.
Карамышева Н. С.

Пенза, 2025

Цель работы

Изучить работу с файлами и механизмы сериализации данных.

Лабораторное задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла (JFileChooser). Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:

1. Добавили 4 новые кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно.

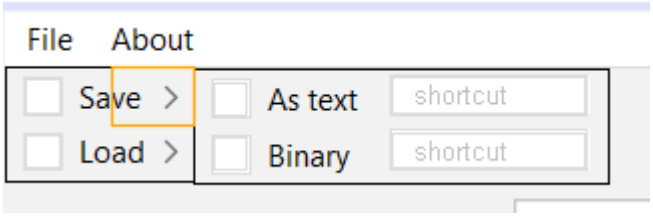

Рисунок 1 - Кнопки для сохранения файла

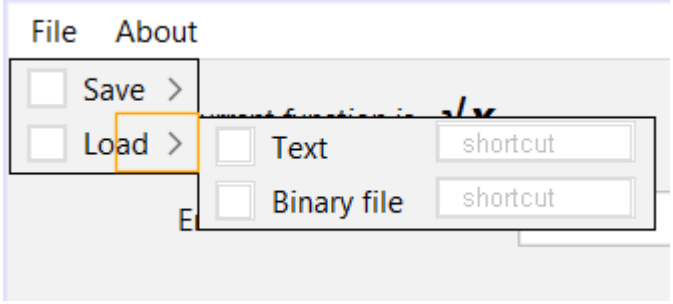

Рисунок 2 - Кнопки для загрузки файла

2. В методах основного класса предусмотрели использование стандартного диалога открытия файла (JFileChooser).

```
private void menuSaveAsTextActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JFileChooser fc = new JFileChooser();

    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text file", "txt");
    fc.setFileFilter(filter);

    int result = fc.showOpenDialog(this);
```

Рисунок 3 - Использование стандартного диалога открытия файла в программе

Результат работы программы:

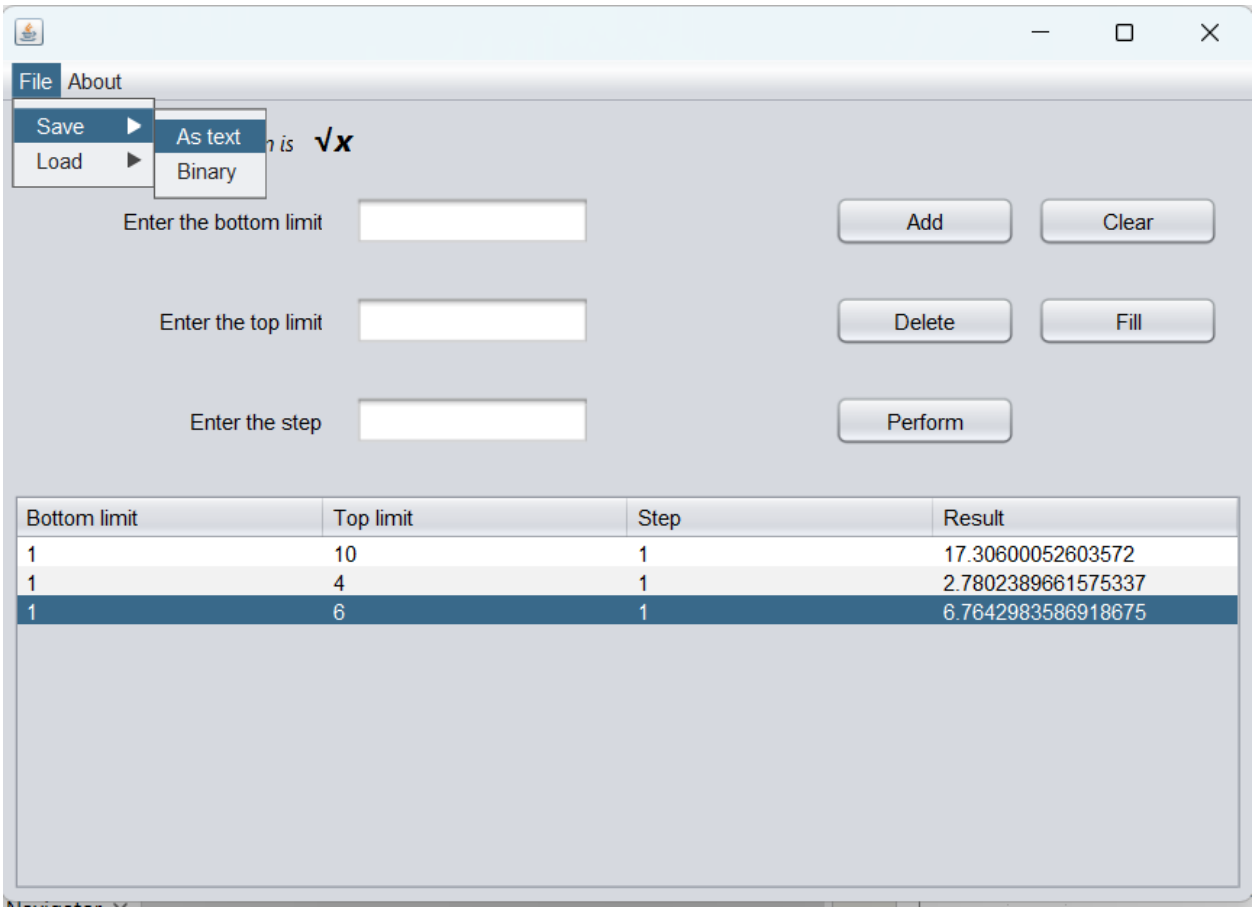

Рисунок 4 - Заполнение таблицы и нажатие на кнопку сохранения файла как текстового

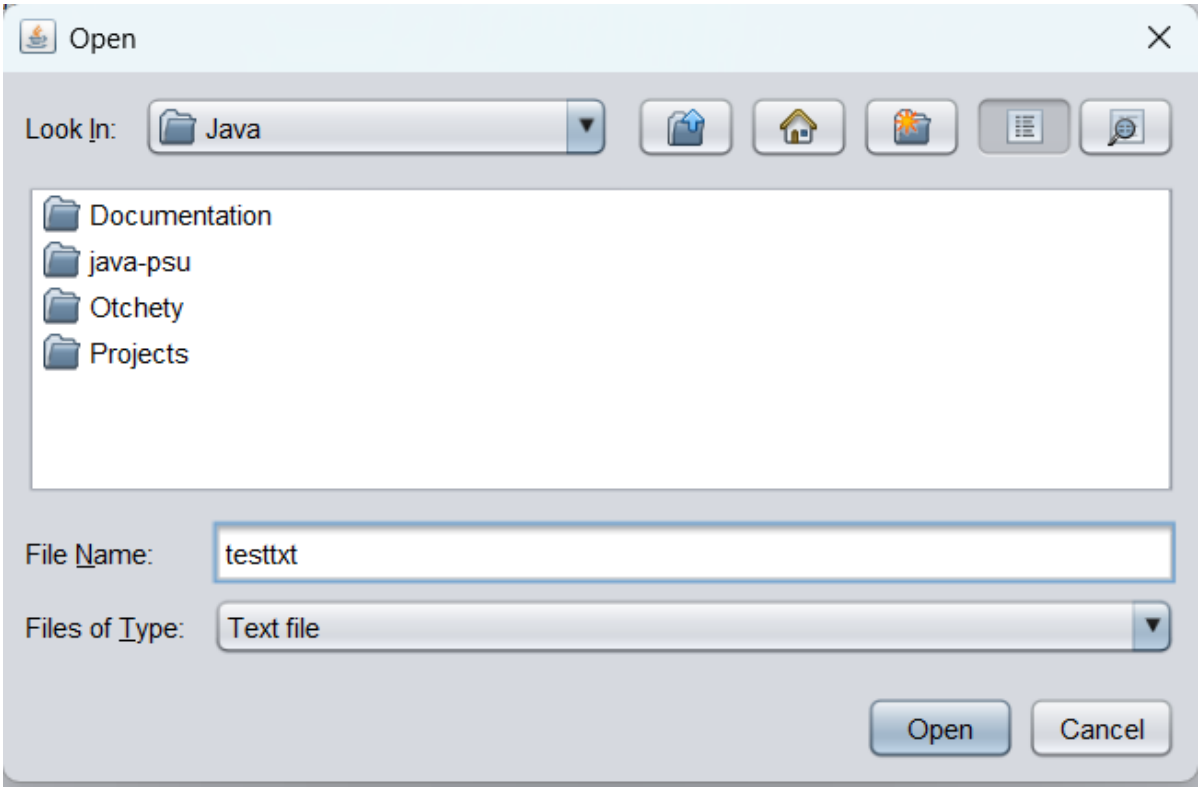

Рисунок 5 - Открытие диалогового окна для сохранения

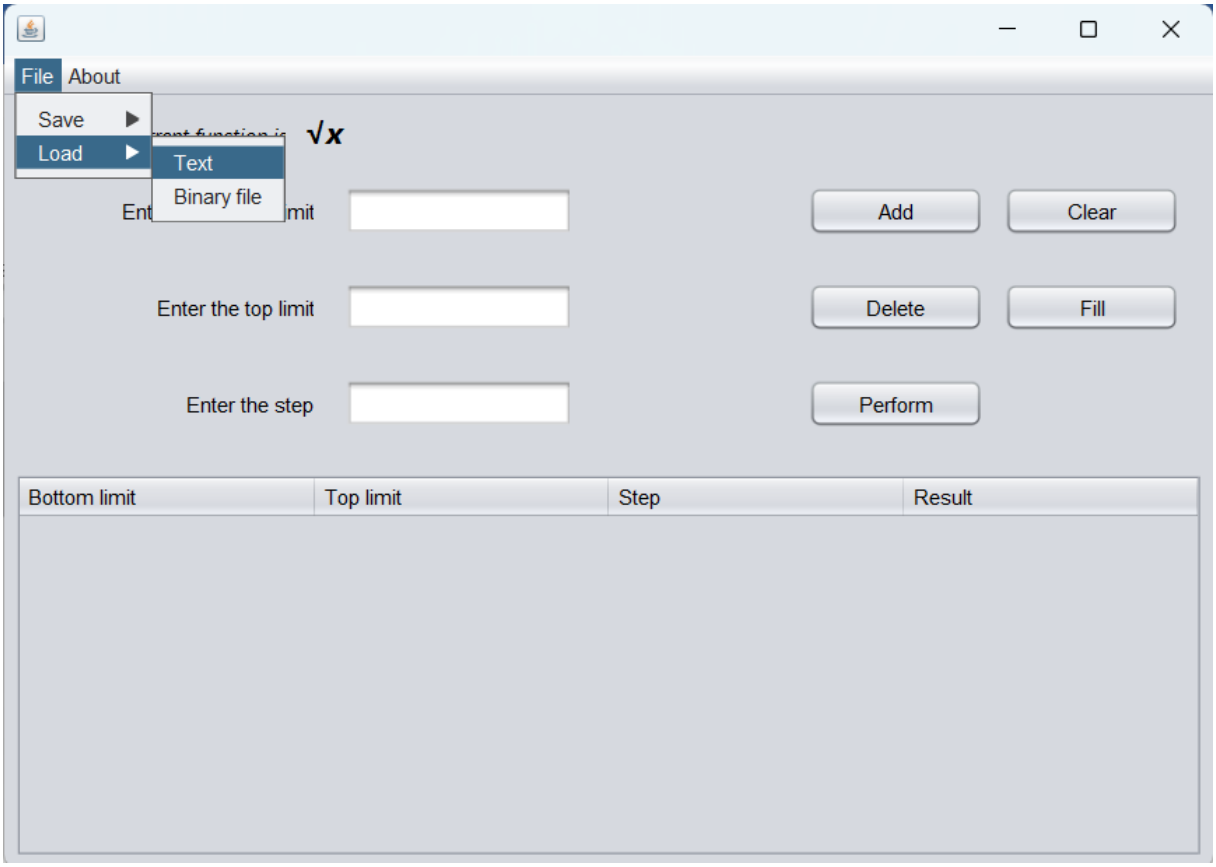

Рисунок 6 - Нажатие кнопки загрузки текстового файла

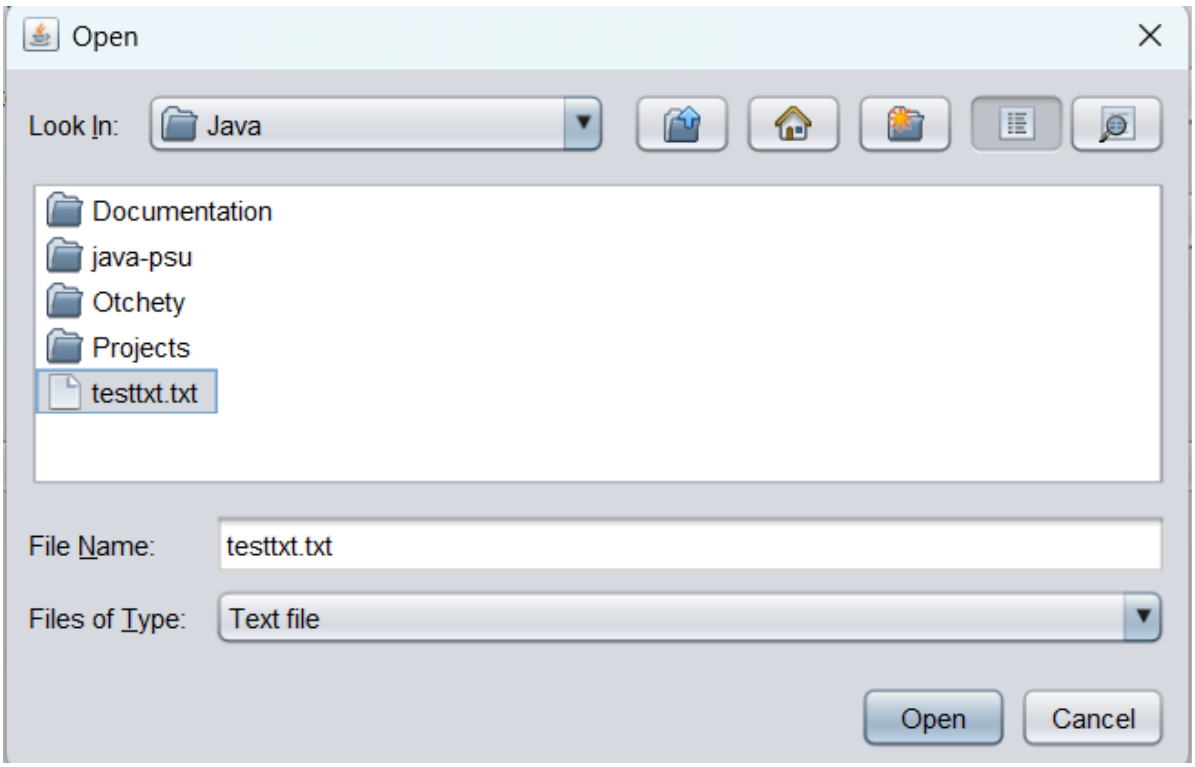

Рисунок 7 - Выбор файла для загрузки

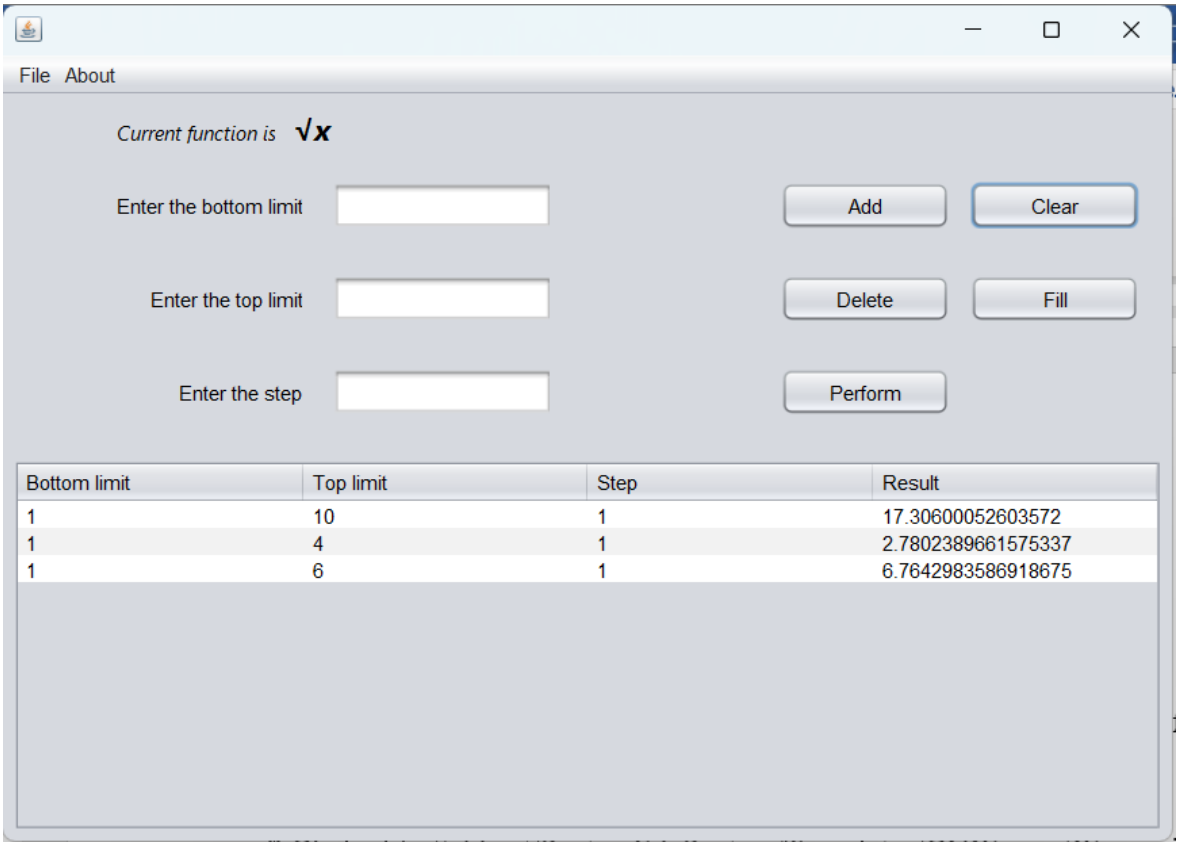

Рисунок 8 - Результат работы загрузки текстового файла

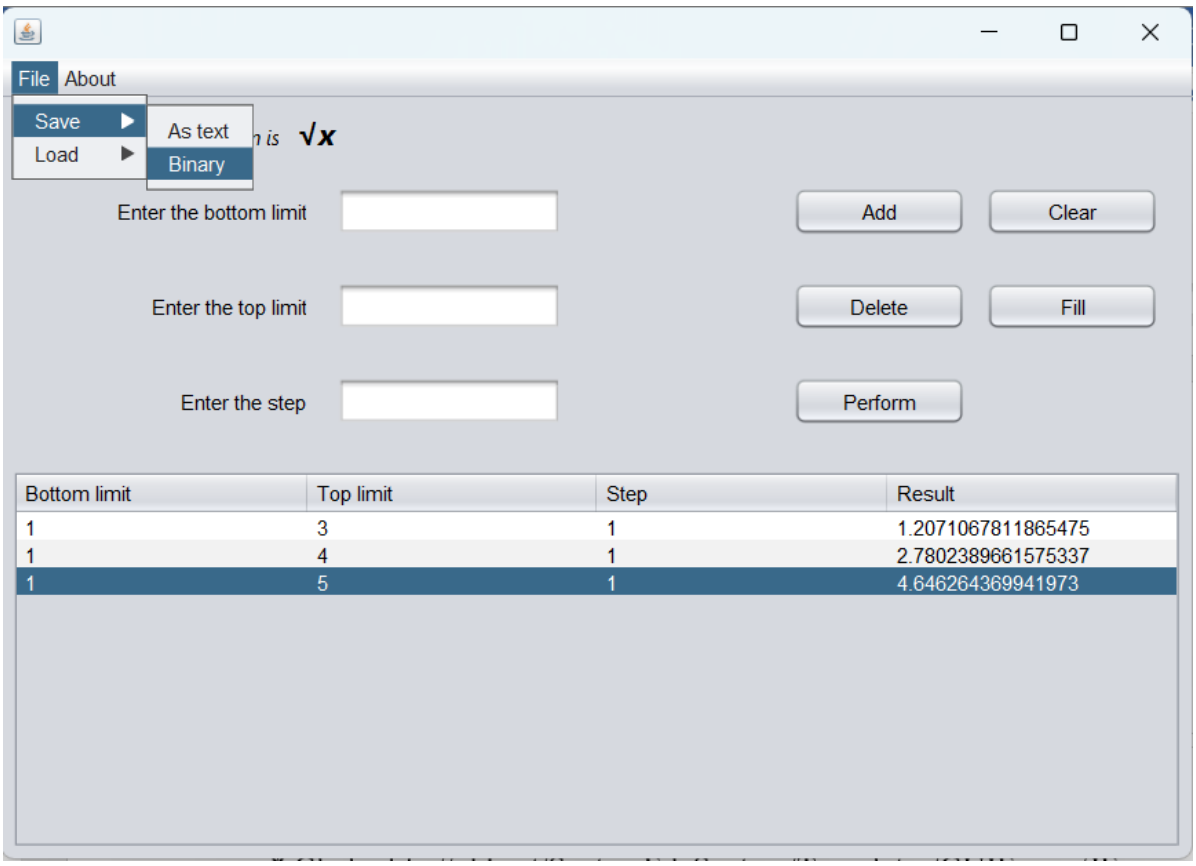

Рисунок 9 - Заполнение таблицы и нажатие на кнопку сохранения файла как бинарного

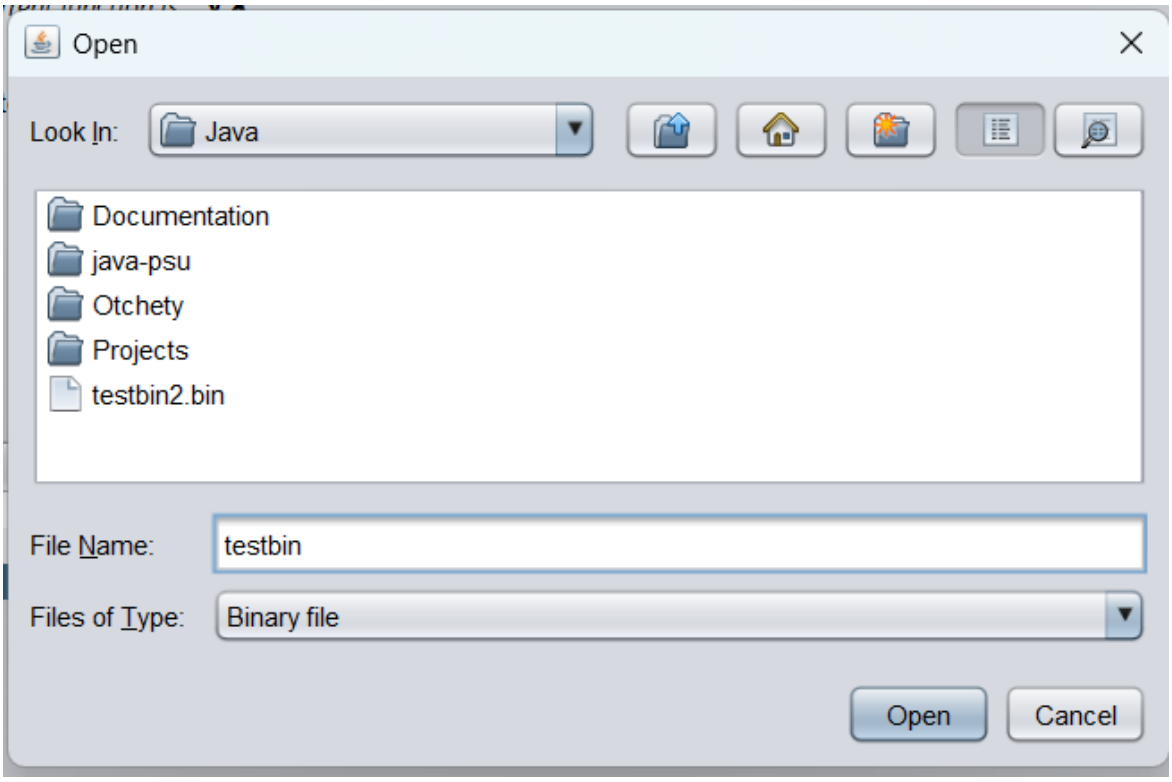

Рисунок 10 - Открытие диалогового окна для сохранения

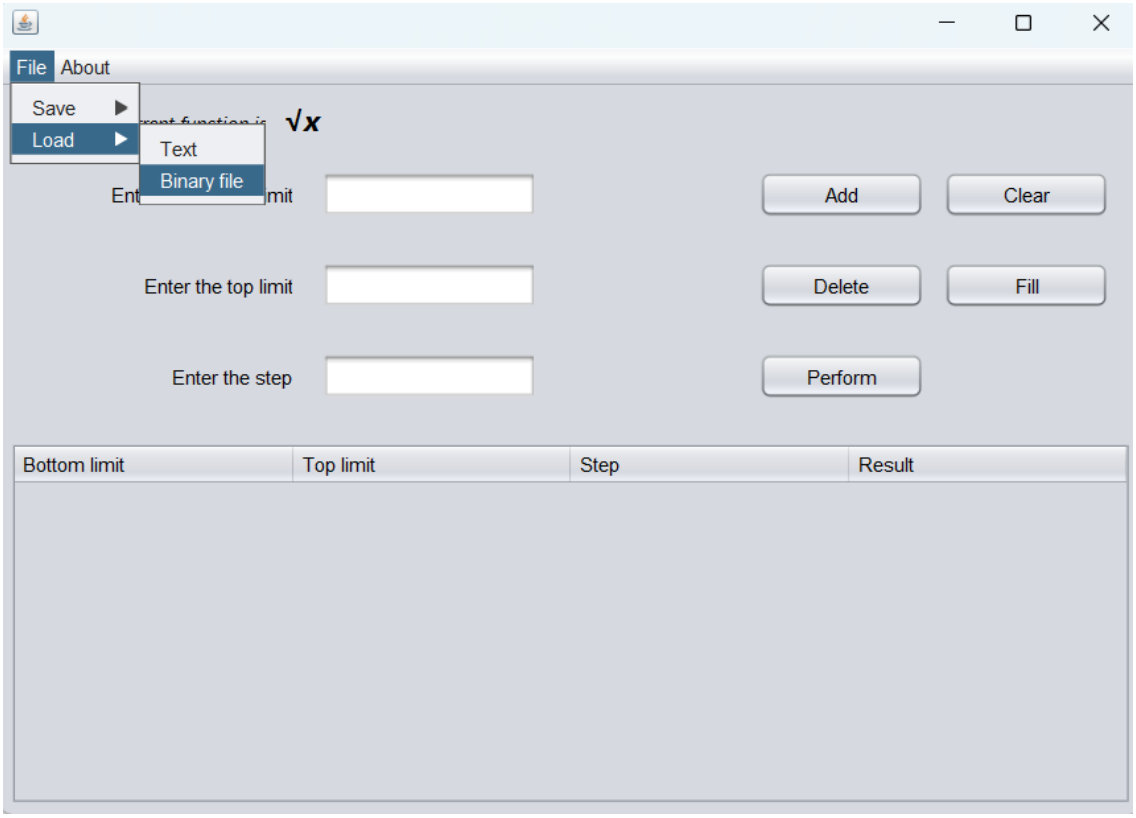

Рисунок 11 - Нажатие кнопки загрузки бинарного файла

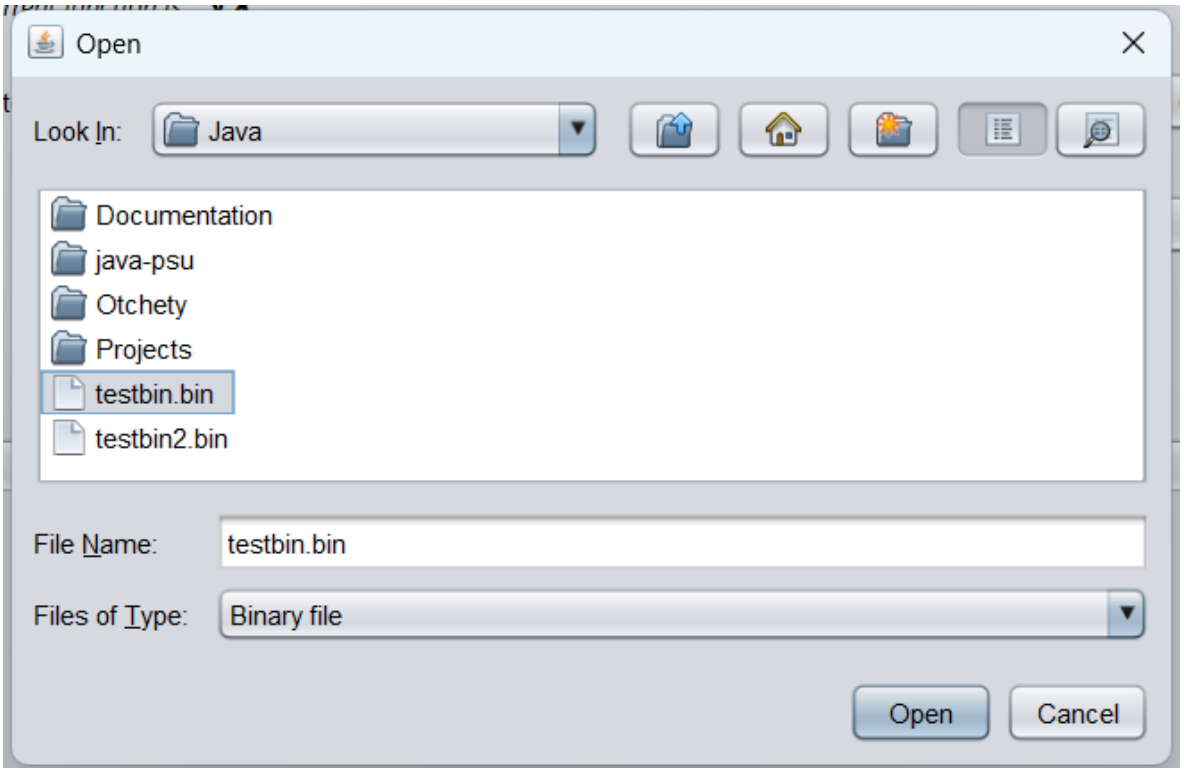

Рисунок 12 - Выбор файла для загрузки

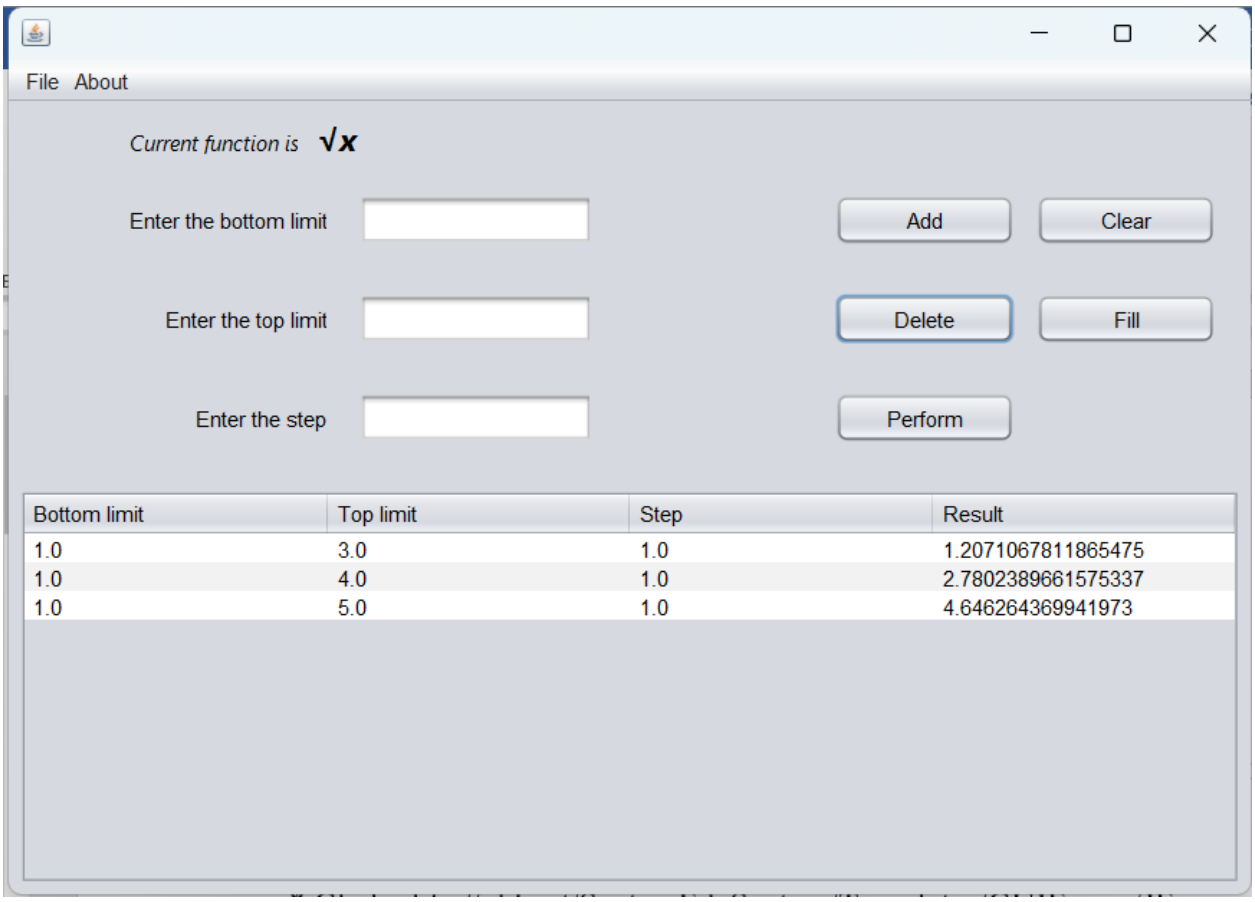

Рисунок 13 - Результат работы загрузки бинарного файла

Вывод

Изучили работу с файлами и механизмы сериализации данных, добавили 4 новые кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно.

Листинг

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package ru.psu.examplefirst;

import java.util.LinkedList;
import java.util.ArrayList;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import java.io.*;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.JFileChooser;
import javax.swing.filechooser.FileNameExtensionFilter;
import com.fasterxml.jackson.databind.ObjectMapper;

/**
 *
```

```
 * @author Asus
 */
public class FourthTask extends javax.swing.JFrame {

    /**
     * Creates new form FirstExample
     */

    LinkedList<RecIntegral> functionList;

    public FourthTask() {
        initComponents();
        functionList = new LinkedList<>();
    }

    boolean isClear = true;

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
```

```
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {

        jPanel = new javax.swing.JPanel();
        jScrollPane1 = new javax.swing.JScrollPane();
        jTable = new javax.swing.JTable();
        txtBot = new javax.swing.JTextField();
        txtTop = new javax.swing.JTextField();
        txtStep = new javax.swing.JTextField();
        labelBot = new javax.swing.JLabel();
        labelTop = new javax.swing.JLabel();
        labelStep = new javax.swing.JLabel();
        btnAdd = new javax.swing.JButton();
        btnDelete = new javax.swing.JButton();
        btnPerform = new javax.swing.JButton();
        labelFunction = new javax.swing.JLabel();
        labelF = new javax.swing.JLabel();
        btnClear = new javax.swing.JButton();
        btnFill = new javax.swing.JButton();
        jMenuBar1 = new javax.swing.JMenuBar();
        menuFile = new javax.swing.JMenu();
        menuSave = new javax.swing.JMenu();
```

```
        menuSaveAsText = new javax.swing.JMenuItem();
        menuSaveBinary = new javax.swing.JMenuItem();
        menuSaveJSON = new javax.swing.JMenuItem();
        menuLoad = new javax.swing.JMenu();
        menuLoadText = new javax.swing.JMenuItem();
        menuLoadBinary = new javax.swing.JMenuItem();
        menuLoadJSON = new javax.swing.JMenuItem();
        menuAbout = new javax.swing.JMenu();


setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

        jTable.setModel(new javax.swing.table.DefaultTableModel(
            new Object [][] {


            },
            new String [] {
                "Bottom limit", "Top limit", "Step", "Result"
            }
        ) {
            boolean[] canEdit = new boolean [] {
                true, true, true, false
            };
```

```
    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
jScrollPane1.setViewportView(jTable);

labelBot.setText("Enter the bottom limit");

labelTop.setText("Enter the top limit");

labelStep.setText("Enter the step");

btnAdd.setText("Add");
btnAdd.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnAddActionPerformed(evt);
    }
});

btnDelete.setText("Delete");
btnDelete.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        btnDeleteActionPerformed(evt);
    }
});

btnPerform.setText("Perform");
btnPerform.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnPerformActionPerformed(evt);
    }
});

labelFunction.setFont(new java.awt.Font("Segoe UI", 2, 12)); // NOI18N
labelFunction.setText("Current function is");

labelF.setFont(new java.awt.Font("Segoe UI", 3, 18)); // NOI18N
labelF.setText(" √x");

btnClear.setText("Clear");
btnClear.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnClearActionPerformed(evt);
    }
});
```

```
        btnFill.setText("Fill");
        btnFill.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnFillActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout jPanelLayout = new javax.swing.GroupLayout(jPanel);
        jPanel.setLayout(jPanelLayout);
        jPanelLayout.setHorizontalGroup(
            jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(jPanelLayout.createSequentialGroup()
                .addContainerGap()
                .addComponent(jScrollPane1)
                .addContainerGap())
            .addGroup(jPanelLayout.createSequentialGroup()
                .addGap(67, 67, 67)
                .addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
                .addGroup(jPanelLayout.createSequentialGroup()

.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)
                        .addComponent(labelStep,
javax.swing.GroupLayout.Alignment.TRAILING)
                        .addComponent(labelBot,
javax.swing.GroupLayout.Alignment.TRAILING)
                        .addComponent(labelTop,
javax.swing.GroupLayout.Alignment.TRAILING))
                    .addGap(18, 18, 18)

.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)
                        .addComponent(txtBot,
javax.swing.GroupLayout.PREFERRED_SIZE,                 130,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(txtTop,
javax.swing.GroupLayout.PREFERRED_SIZE,                 130,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(txtStep,
javax.swing.GroupLayout.PREFERRED_SIZE,                 130,
javax.swing.GroupLayout.PREFERRED_SIZE)))
                .addGroup(jPanelLayout.createSequentialGroup()
                    .addComponent(labelFunction)
```

```
.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(labelF,
javax.swing.GroupLayout.PREFERRED_SIZE,                    37,
javax.swing.GroupLayout.PREFERRED_SIZE)))
            .addGap(135, 135, 135)

.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING, false)
                .addComponent(btnAdd,  javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(btnDelete,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(btnPerform,
javax.swing.GroupLayout.DEFAULT_SIZE, 100, Short.MAX_VALUE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)

.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING, false)
                .addComponent(btnClear,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(btnFill,    javax.swing.GroupLayout.DEFAULT_SIZE,
100, Short.MAX_VALUE))
```

```
            .addContainerGap(20, Short.MAX_VALUE))
        );
        jPanelLayout.setVerticalGroup(
jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING
)
            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelLayout.createSequentialGroup()
                .addGap(8, 8, 8)
.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASELINE)
                    .addComponent(labelFunction)
                    .addComponent(labelF))
                .addGap(19, 19, 19)
.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASELINE)
                    .addComponent(txtBot,
javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(labelBot)
                    .addComponent(btnAdd)
                    .addComponent(btnClear))
```

```
            .addGap(27, 27, 27)

.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASELINE)
                .addComponent(txtTop,
javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(labelTop)
                .addComponent(btnDelete)
                .addComponent(btnFill))
            .addGap(27, 27, 27)

.addGroup(jPanelLayout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASELINE)
                .addComponent(txtStep,
javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(labelStep)
                .addComponent(btnPerform))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,  26,
Short.MAX_VALUE)
```

```
            .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,                                    220,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap())
        );

        menuFile.setText("File");

        menuSave.setText("Save");

        menuSaveAsText.setText("As text");
        menuSaveAsText.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                menuSaveAsTextActionPerformed(evt);
            }
        });
        menuSave.add(menuSaveAsText);

        menuSaveBinary.setText("Binary");
        menuSaveBinary.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                menuSaveBinaryActionPerformed(evt);
            }
```

```
});
menuSave.add(menuSaveBinary);

menuSaveJSON.setText("JSON");
menuSaveJSON.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        menuSaveJSONActionPerformed(evt);
    }
});
menuSave.add(menuSaveJSON);

menuFile.add(menuSave);

menuLoad.setText("Load");

menuLoadText.setText("Text");
menuLoadText.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        menuLoadTextActionPerformed(evt);
    }
});
menuLoad.add(menuLoadText);
```

```
menuLoadBinary.setText("Binary file");
menuLoadBinary.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        menuLoadBinaryActionPerformed(evt);
    }
});
menuLoad.add(menuLoadBinary);

menuLoadJSON.setText("JSON");
menuLoadJSON.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        menuLoadJSONActionPerformed(evt);
    }
});
menuLoad.add(menuLoadJSON);

menuFile.add(menuLoad);

jMenuBar1.add(menuFile);

menuAbout.setText("About");
jMenuBar1.add(menuAbout);
```

```
        setJMenuBar(jMenuBar1);

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jPanel, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jPanel, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
        );

        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void btnAddActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_btnAddActionPerformed
```

```
        // TODO add your handling code here:
        DefaultTableModel model = (DefaultTableModel)jTable.getModel();



        try {



            if (isClear){
                model.addRow(new Object[]{txtBot.getText(), txtTop.getText(), txtStep.getText()});
                functionList.add(new RecIntegral(
                        Double.parseDouble(txtBot.getText()),
                        Double.parseDouble(txtTop.getText()),
                        Double.parseDouble(txtStep.getText())
                ));
            } else {
                JOptionPane.showMessageDialog(null, "Table is clear, you can't "
                        + "entries until the button \'Fill\' has been clicked", "WARNING!!!", JOptionPane.WARNING_MESSAGE);
            }



        } catch (InvalidValueException e){
```

```
            JOptionPane.showMessageDialog(null, e.getMessage() + e.getNumber(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
            model.removeRow(model.getRowCount() - 1);

        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Invalid Value", "ERROR!!!", JOptionPane.ERROR_MESSAGE);
            model.removeRow(model.getRowCount() - 1);

        } finally {
            txtBot.setText("");
            txtTop.setText("");
            txtStep.setText("");

        }
    }//GEN-LAST:event_btnAddActionPerformed

    private void btnDeleteActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_btnDeleteActionPerformed
        // TODO add your handling code here:
        DefaultTableModel model = (DefaultTableModel)jTable.getModel();
        try {
            int selectedRowID = jTable.getSelectedRow();
```

```
            model.removeRow(selectedRowID);
            functionList.remove(selectedRowID);
        } catch (Exception e){
            JOptionPane.showMessageDialog(null, e);
        }

    }//GEN-LAST:event_btnDeleteActionPerformed

    private void btnPerformActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_btnPerformActionPerformed
        // TODO add your handling code here:
        DefaultTableModel model = (DefaultTableModel)jTable.getModel();

        double result;
        int selectedRowID = jTable.getSelectedRow();

        RecIntegral integral = functionList.get(selectedRowID);
        functionList.set(selectedRowID,
integral).setBotLimit(Double.parseDouble(jTable.getValueAt(selectedRowID,
0).toString()));
        functionList.set(selectedRowID,
integral).setTopLimit(Double.parseDouble(jTable.getValueAt(selectedRowID,
1).toString()));
```

```
        functionList.set(selectedRowID,
integral).setStep(Double.parseDouble(jTable.getValueAt(selectedRowID,
2).toString()));



        result = integral.Integral();
        model.setValueAt(result, selectedRowID, 3);
        functionList.set(selectedRowID,
integral).setResult(Double.parseDouble(jTable.getValueAt(selectedRowID,
3).toString()));

    }//GEN-LAST:event_btnPerformActionPerformed

    private void btnClearActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnClearActionPerformed
        // TODO add your handling code here:
        DefaultTableModel model = (DefaultTableModel)jTable.getModel();
        model.setRowCount(0);
        isClear = false;
    }//GEN-LAST:event_btnClearActionPerformed

    private void btnFillActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnFillActionPerformed
        // TODO add your handling code here:
```

```
        DefaultTableModel model = (DefaultTableModel)jTable.getModel();
        Object[] data = new Object[4];
        for (int i = 0; i < functionList.size(); i++){
            data[0] = functionList.get(i).getBotLimit();
            data[1] = functionList.get(i).getTopLimit();
            data[2] = functionList.get(i).getStep();
            data[3] = functionList.get(i).getResult();
            model.addRow(data);
        }
          isClear = true;
    }//GEN-LAST:event_btnFillActionPerformed

    private void menuSaveAsTextActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_menuSaveAsTextActionPerformed
        // TODO add your handling code here:
        JFileChooser fc = new JFileChooser();

        FileNameExtensionFilter filter = new FileNameExtensionFilter("Text file", "txt");
        fc.setFileFilter(filter);

        int result = fc.showSaveDialog(this);
```

```
        if (result == JFileChooser.APPROVE_OPTION){
            File file = fc.getSelectedFile();

            String path = file.getAbsolutePath();
            if (!path.endsWith(".txt"))
                file = new File(path + ".txt");

            try (FileWriter fw = new FileWriter(file);
                BufferedWriter bw = new BufferedWriter(fw)){



                for (int i = 0; i < jTable.getRowCount(); i++){
                    if (jTable.getValueAt(i, 3) == null)
                        throw new InvalidValueException("The result field must be filled in!!!");

                    for (int j = 0; j < jTable.getColumnCount(); j++){
                        bw.write(jTable.getValueAt(i, j).toString() + " ");
                    }
                    bw.newLine();
                }

            } catch (InvalidValueException e) {
```

```
            if (file.exists())
                file.delete();
            JOptionPane.showMessageDialog(null, e.getMessage(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
        } catch (IOException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        }

    }
}//GEN-LAST:event_menuSaveAsTextActionPerformed

private void menuLoadTextActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_menuLoadTextActionPerformed
    // TODO add your handling code here:
    JFileChooser fc = new JFileChooser();

    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text file", "txt");
    fc.setFileFilter(filter);

    int result = fc.showOpenDialog(this);

    if (result == JFileChooser.APPROVE_OPTION){
```

```
File file = fc.getSelectedFile();

String path = file.getAbsolutePath();
if (!path.endsWith(".txt"))
   file = new File(path + ".txt");

try {
   FileReader fr = new FileReader(file);
   BufferedReader br = new BufferedReader(fr);

   DefaultTableModel model = (DefaultTableModel)jTable.getModel();
   if (model.getRowCount() != 0)
      throw new InvalidValueException("You cannot fill a table while it contains value!!!");
   if (!isClear)
      throw new InvalidValueException("You cannot fill a table while it is cleared!!!");

   Object[] lines = br.lines().toArray();

   for (int i = 0; i < lines.length; i++){
      String[] row = lines[i].toString().split(" ");
      model.addRow(row);
```

```
            }


            br.close();
            fr.close();


        } catch (FileNotFoundException ex) {
            JOptionPane.showMessageDialog(null, "File not found!!!", "ERROR!!!", JOptionPane.ERROR_MESSAGE);
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (IOException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (InvalidValueException ex) {
            JOptionPane.showMessageDialog(null, ex.getMessage(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
        }
      }
    }//GEN-LAST:event_menuLoadTextActionPerformed


    private void menuSaveBinaryActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_menuSaveBinaryActionPerformed
        // TODO add your handling code here:
        JFileChooser fc = new JFileChooser();
```

```
        FileNameExtensionFilter filter = new FileNameExtensionFilter("Binary file", "bin");
        fc.setFileFilter(filter);

        int result = fc.showSaveDialog(this);

        if (result == JFileChooser.APPROVE_OPTION){
            File file = fc.getSelectedFile();

            String path = file.getAbsolutePath();
            if (!path.endsWith(".bin"))
                file = new File(path + ".bin");

            try (FileOutputStream fos = new FileOutputStream(file);
                 ObjectOutputStream oos = new ObjectOutputStream(fos);) {

                for (int i = 0; i < jTable.getRowCount(); i++){
                    if (jTable.getValueAt(i, 3) == null)
                        throw new InvalidValueException("The result field must be filled in!!!");

                    oos.writeObject(new RecIntegral(
```

```
                    Double.parseDouble(jTable.getValueAt(i, 0).toString()),
                    Double.parseDouble(jTable.getValueAt(i, 1).toString()),
                    Double.parseDouble(jTable.getValueAt(i, 2).toString()),
                    Double.parseDouble(jTable.getValueAt(i, 3).toString())
                ));
            }


        } catch (FileNotFoundException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (NumberFormatException | IOException ex){
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (InvalidValueException e) {
            if (file.exists())
                file.delete();
            JOptionPane.showMessageDialog(null, e.getMessage(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
        }
    }
    }//GEN-LAST:event_menuSaveBinaryActionPerformed
```

```
    private void menuLoadBinaryActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_menuLoadBinaryActionPerformed
        // TODO add your handling code here:
        JFileChooser fc = new JFileChooser();

        FileNameExtensionFilter filter = new FileNameExtensionFilter("Binary file", "bin");
        fc.setFileFilter(filter);

        int result = fc.showOpenDialog(this);

        if (result == JFileChooser.APPROVE_OPTION){
            File file = fc.getSelectedFile();

            String path = file.getAbsolutePath();
            if (!path.endsWith(".bin"))
                file = new File(path + ".bin");

            try {
                FileInputStream fis = new FileInputStream(file);
                ObjectInputStream ois = new ObjectInputStream(fis);

                DefaultTableModel model = (DefaultTableModel)jTable.getModel();
```

```
            if (model.getRowCount() != 0)
                throw new InvalidValueException("You cannot fill a table while it contains value!!!");
            if (!isClear)
                throw new InvalidValueException("You cannot fill a table while it is cleared!!!");

            while (fis.available() > 0) {
                RecIntegral integral = (RecIntegral) ois.readObject();

                model.addRow(new Object[]{
                    integral.getBotLimit(),
                    integral.getTopLimit(),
                    integral.getStep(),
                    integral.getResult()
                });
            }

        } catch (FileNotFoundException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (ClassNotFoundException | IOException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
```

```
            } catch (InvalidValueException ex) {
                JOptionPane.showMessageDialog(null, ex.getMessage(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
            }

        }
    }//GEN-LAST:event_menuLoadBinaryActionPerformed

    private void menuSaveJSONActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_menuSaveJSONActionPerformed
        // TODO add your handling code here:
        JFileChooser fc = new JFileChooser();

        FileNameExtensionFilter filter = new FileNameExtensionFilter("JSON file", "json");
        fc.setFileFilter(filter);

        int result = fc.showSaveDialog(this);

        if (result == JFileChooser.APPROVE_OPTION){
            File file = fc.getSelectedFile();

            String path = file.getAbsolutePath();
```

```
        if (!path.endsWith(".json"))
            file = new File(path + ".json");

        try {
            //StringWriter sw = new StringWriter();
            ObjectMapper mapper = new ObjectMapper();

mapper.enable(com.fasterxml.jackson.databind.SerializationFeature.INDENT_OUTPUT);

            List<RecIntegral> list = new ArrayList();

            for (int i = 0; i < jTable.getRowCount(); i++){
                if (jTable.getValueAt(i, 3) == null)
                    throw new InvalidValueException("The result field must be filled in!!!");

                RecIntegral integral = new RecIntegral(
                        Double.parseDouble(jTable.getValueAt(i, 0).toString()),
                        Double.parseDouble(jTable.getValueAt(i, 1).toString()),
                        Double.parseDouble(jTable.getValueAt(i, 2).toString()),
                        Double.parseDouble(jTable.getValueAt(i, 3).toString())
                );
```

```
                list.add(integral);
            }

            mapper.writeValue(file, list);

        } catch (FileNotFoundException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (InvalidValueException e){
            if (file.exists())
                file.delete();
            JOptionPane.showMessageDialog(null, e.getMessage(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
        } catch (IOException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}//GEN-LAST:event_menuSaveJSONActionPerformed

private void menuLoadJSONActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_menuLoadJSONActionPerformed
    // TODO add your handling code here:
    JFileChooser fc = new JFileChooser();
```

```
        FileNameExtensionFilter filter = new FileNameExtensionFilter("JSON file", "json");
        fc.setFileFilter(filter);

        int result = fc.showOpenDialog(this);

        if (result == JFileChooser.APPROVE_OPTION){
            File file = fc.getSelectedFile();

            String path = file.getAbsolutePath();
            if (!path.endsWith(".json"))
                file = new File(path + ".json");

            try {
                ObjectMapper mapper = new ObjectMapper();

                RecIntegral[] integral = mapper.readValue(file, RecIntegral[].class);

                DefaultTableModel model = (DefaultTableModel)jTable.getModel();

                model.setRowCount(0);
```

```
            functionList.clear();


            for (RecIntegral rec : integral){
                model.addRow(new Object[]{
                    rec.getBotLimit(),
                    rec.getTopLimit(),
                    rec.getStep(),
                    rec.getResult()
                });
                functionList.add(rec);
            }



        } catch (FileNotFoundException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } catch (IOException ex) {
            Logger.getLogger(FourthTask.class.getName()).log(Level.SEVERE, null, ex);
        } //catch (InvalidValueException ex) {
          //   JOptionPane.showMessageDialog(null, ex.getMessage(), "ERROR!!!", JOptionPane.ERROR_MESSAGE);
        //}
```

```
        }

    }//GEN-LAST:event_menuLoadJSONActionPerformed


    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
```

```
            }
        } catch (ClassNotFoundException ex) {
java.util.logging.Logger.getLogger(FourthTask.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
        } catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(FourthTask.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
        } catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(FourthTask.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(FourthTask.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
        }
        //</editor-fold>
        //</editor-fold>
        //</editor-fold>
        //</editor-fold>
        //</editor-fold>
        //</editor-fold>
        //</editor-fold>
```

```
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new FourthTask().setVisible(true);
        }
    });
}


// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton btnAdd;
private javax.swing.JButton btnClear;
private javax.swing.JButton btnDelete;
private javax.swing.JButton btnFill;
private javax.swing.JButton btnPerform;
private javax.swing.JMenuBar jMenuBar1;
private javax.swing.JPanel jPanel;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable;
private javax.swing.JLabel labelBot;
private javax.swing.JLabel labelF;
```

```
    private javax.swing.JLabel labelFunction;
    private javax.swing.JLabel labelStep;
    private javax.swing.JLabel labelTop;
    private javax.swing.JMenu menuAbout;
    private javax.swing.JMenu menuFile;
    private javax.swing.JMenu menuLoad;
    private javax.swing.JMenuItem menuLoadBinary;
    private javax.swing.JMenuItem menuLoadJSON;
    private javax.swing.JMenuItem menuLoadText;
    private javax.swing.JMenu menuSave;
    private javax.swing.JMenuItem menuSaveAsText;
    private javax.swing.JMenuItem menuSaveBinary;
    private javax.swing.JMenuItem menuSaveJSON;
    private javax.swing.JTextField txtBot;
    private javax.swing.JTextField txtStep;
    private javax.swing.JTextField txtTop;
    // End of variables declaration//GEN-END:variables
}
```